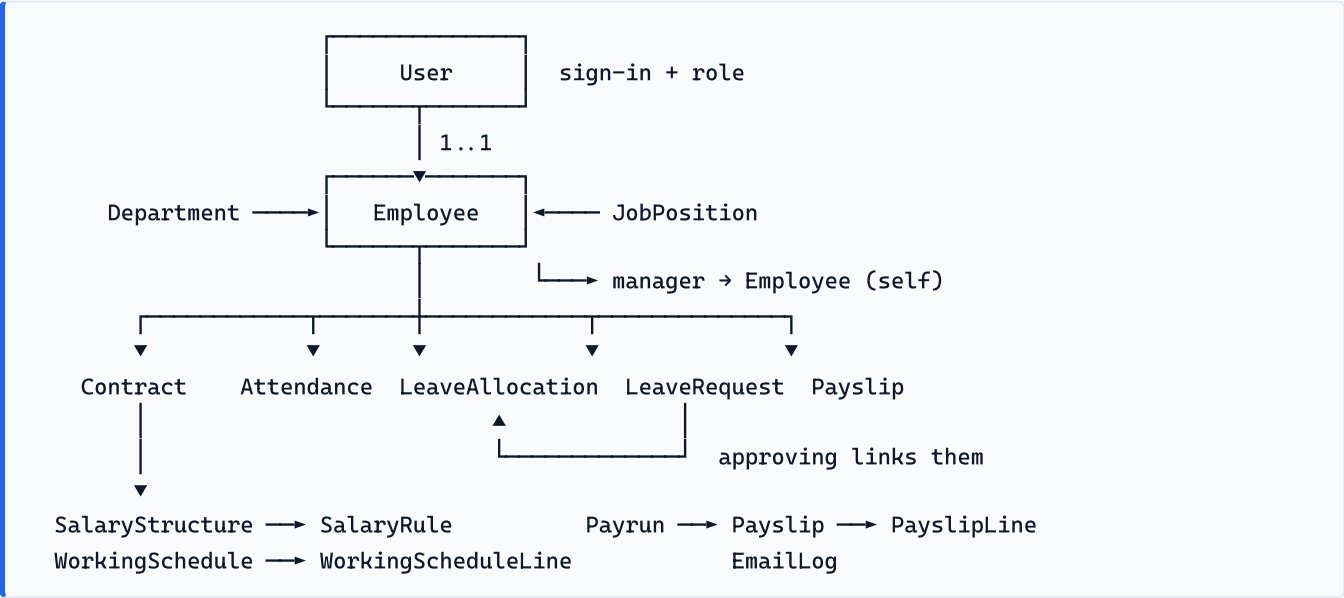


Data model

PostgreSQL 16 through Prisma 6. The schema is `api/prisma/schema.prisma` ; this explains the shape it describes and the rules the services enforce on top of it.

The shape



The employee record is the hub. Everything daily hangs off it, and payroll reads through it.

- A `User` is a sign-in with a role, and it always has exactly one `Employee` . One person is one record and one account, never one without the other: creating an employee creates the account in the same transaction, and an admin is an employee like everyone else. There is nothing to link, so nothing can drift out of sync.
- `mustChangePassword` marks an account still on a password the system issued. `invitedAt` records when the invite carrying it actually went out; it stays null when the send failed, so "was this person ever asked to sign in" has an honest answer.
- An `Employee` may manage other employees — `manager` is a self-relation.
- Deleting an employee cascades to attendance, allocations, requests and contracts, and takes the account with it. Payslips do not cascade, which is why the API deactivates both instead of deleting once payroll history exists.

Ids, money and time

Concern	Choice	Why
Primary keys	<code>uuid(7)</code>	UUIDv7: globally unique but time-sortable , unlike UUIDv4, so index locality stays good. Validated by <code>IsEntityId</code> , not <code>IsUUID</code> — see architecture.md .

Concern	Choice	Why
Money	<code>Decimal(12,2)</code>	A float cannot represent a rupee exactly. Totals reconcile to the cent.
Hours	<code>Decimal(8,2)</code>	Same reason; half-days and breaks are fractional.
Timestamps	<code>DateTime</code>	Stored as instants. Date-only columns are read in UTC so server and client agree.

Prisma surfaces `Decimal`, which is normalised to a plain number at the API boundary by `common/decimal.ts`, so a JSON consumer never has to know.

Contracts are period-scoped

An employee holds many contracts over time. Payroll does not take "the employee's contract" — it resolves the single `RUNNING` contract whose `dateStart` / `dateEnd` range overlaps the payroll period. Expired contracts are never used, which is what makes a mid-year raise or a role change compute correctly for each period rather than retroactively.

Creating a `RUNNING` contract that overlaps another for the same employee is rejected at save time, because two applicable contracts would make the resolution ambiguous.

A contract carries its own `jobPosition`, `workingSchedule` and `salaryStructure`, each of which **overrides** the employee's when payroll runs. That is how someone can move to a four-day week for one contract period without editing their employee record.

Hours are derived, never stored from a client

`WorkingSchedule.hoursPerWeek` is a stored column, but it is always computed from the `WorkingScheduleLine` rows — the `{ dayOfWeek, startTime, endTime, breakHours }` weekly pattern — by a pure function in `shared/src/domain.ts`. The API recomputes it on every write and ignores any value a client sends.

Overnight shifts roll over correctly: a line running 22:00 → 06:00 is eight hours, not minus sixteen.

Period working days come from the schedule too, not from a flat 30-day assumption, so a day rate means what it says.

Attendance is derived from punches

An `Attendance` row stores `checkIn` and optionally `checkOut`. Everything else — `workedHours`, `overtimeHours` and `status` — is derived from those two against that weekday's schedule line. Late arrival, half day and missing check-out are detected rather than entered.

A correction may override `status` explicitly. Doing so sets `manuallyEdited` and records `editedById`, `editedAt` and `editReason`, so a changed record always says who changed it and why.

Leave balance is derived, never stored

There is no balance column. For a given employee and type:

```
allocated =  $\Sigma$  approved allocations valid on the date
taken     =  $\Sigma$  approved requests
pending   =  $\Sigma$  requests awaiting approval
remaining = allocated - taken
```

Approving a request links it to a specific `LeaveAllocation`, so consumption is auditable rather than implied by arithmetic.

A request is refused when it overlaps existing leave, exceeds the remaining balance, or breaches the type's `maxDaysPerRequest`. Duration counts only days the employee was scheduled to work, so a Friday-to-Monday absence is two days for a Monday-to-Friday schedule.

A `TimeOffType` decides the policy: its `unit` (`DAY` or `HOURL`), whether it `requiresAllocation`, whether it `requiresApproval`, and whether it is `paid`. Unpaid leave needs no allocation, which is why its remaining figure is not a limit.

The payroll engine

`api/src/modules/payroll/payroll-engine.service.ts`.

A `SalaryStructure` holds an ordered set of `SalaryRule`s. Computing a payslip walks them in ascending `sequence`, and **each result enters scope under its own code**, so a later rule can build on an earlier subtotal:

```
BASIC    fixed / from the contract wage
HRA      percentage of BASIC
GROSS    formula: BASIC + HRA + ...
PF       percentage of BASIC
TDS      formula, conditional
NET      formula: GROSS - PF - PT - TDS - ULD
```

`NET` is literally that expression, not a hardcoded sum. Change the rules and the arithmetic changes with them.

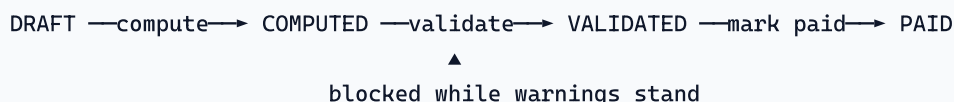
A rule computes one of three ways:

computeType	Uses
<code>FIXED</code>	<code>amountFixed</code>
<code>PERCENTAGE</code>	<code>amountPercentage</code> of <code>percentageBase</code> (another rule's code)
<code>FORMULA</code>	<code>formula</code> , evaluated against the codes already in scope

An optional `condition` skips the rule entirely when it evaluates false. Formulas are validated before being stored, and evaluate in a restricted scope: dangerous identifiers are rejected before evaluation and globals are shadowed, so a formula cannot reach the host environment.

`category` decides the sign — `DEDUCTION` and `CONTRIBUTION` reduce net pay, everything else adds — and whether a line appears on the printed payslip.

The pay run lifecycle



Creating a run needs the period, the structure and an explicit list of employees. Eligibility is a separate read (`GET /payruns/eligible-employees`) that returns everyone in scope with a reason when they cannot be paid — no applicable contract, say — so the person building the run sees why before committing to it.

Warnings are surfaced before validation and **block** it:

- a duplicate payslip covering the same period for the same employee
- missing bank details
- no applicable contract for the period
- negative net pay

`send-payslips` generates each PDF and records every attempt in `EmailLog` with its status and, on failure, the error. With no `SMTP_HOST` configured it records the attempt instead of dialling out, so the flow is demonstrable without credentials.

Enumerations

All in `shared/src/enums.ts` as plain const objects rather than TypeScript `enum` s, so the values survive JSON transport unchanged and can be iterated to build a select.

Enum	Values
<code>Role</code>	<code>EMPLOYEE</code> <code>HR_MANAGER</code> <code>HR_PAYROLL_USER</code> <code>HR_PAYROLL_MANAGER</code> <code>ADMIN</code>
<code>EmployeeType</code>	<code>FULL_TIME</code> <code>PART_TIME</code> <code>CONTRACT</code> <code>INTERN</code>
<code>EmployeeStatus</code>	<code>ACTIVE</code> <code>ON_LEAVE</code> <code>INACTIVE</code>
<code>ContractStatus</code>	<code>DRAFT</code> <code>RUNNING</code> <code>EXPIRED</code> <code>CANCELLED</code>
<code>ContractType</code>	<code>PERMANENT</code> <code>FIXED_TERM</code> <code>INTERNSHIP</code> <code>FREELANCE</code>
<code>ScheduleType</code>	<code>FULL_TIME</code> <code>PART_TIME</code> <code>FLEXIBLE</code>
<code>AttendanceStatus</code>	<code>PRESENT</code> <code>LATE</code> <code>ABSENT</code> <code>MISSING_CHECKOUT</code> <code>HALF_DAY</code>

Enum	Values
LeaveUnit	DAY HOUR
LeaveRequestStatus	DRAFT TO_APPROVE APPROVED REFUSED CANCELLED
AllocationStatus	DRAFT APPROVED REFUSED
RuleCategory	BASIC ALLOWANCE GROSS DEDUCTION CONTRIBUTION NET
ComputeType	FIXED PERCENTAGE FORMULA
PayrunStatus / PayslipStatus	DRAFT COMPUTED VALIDATED PAID CANCELLED
EmailStatus	QUEUED SENT FAILED

The web client maps every one of these to a label and a tone in `frontend/src/lib/status.ts`, so a status reads the same wherever it appears.

Migrations and seed

```
npm run db:migrate    # create and apply
npm run db:seed       # seed
npm run db:reset      # drop, re-migrate, reseed
npm run db:studio     # browse
```

The seed is deliberately imperfect so the warning paths are demonstrable: two employees have no bank details, several attendance records are missing a check-out or were manually corrected, and some contracts expire within 30 days.

`20260905170408_one_identity_per_person` is hand-written rather than generated, because making `Employee.userId` required on a database that already holds payroll history is not a change Prisma can make on its own. It backfills in both directions before adding the constraint: an account for every employee that lacked one, and an employee for every account that lacked one, continuing the existing `EMP####` sequence rather than restarting it. Nothing is deleted.

The accounts it creates are inactive and carry a bcrypt digest of a value nobody holds, so the migration cannot hand anyone a way in as a side effect. `POST /employees/:id/reinvite` is how they are brought online.